

Explorador y Generador de Laberintos en C

Mario Matos

Profesor: Jesús Urbano

1. Introducción

El presente proyecto consiste en el diseño y desarrollo de un programa en lenguaje C que automatiza la creación, jugabilidad y resolución de laberintos bidimensionales. A través de este sistema, se explora la integración práctica de estructuras de datos fundamentales y la teoría de grafos, utilizando lenguajes de programación estructurados. El programa ofrece una experiencia interactiva que combina la generación procedimental de caminos con entornos lúdicos, permitiendo en primer lugar el entretenimiento del usuario mediante control manual, para posteriormente demostrar la eficiencia del cálculo computacional en la búsqueda de rutas óptimas.

2. Planteamiento del Problema

En el desarrollo de software y sistemas autónomos, el desplazamiento a través de mapas bidimensionales presenta dos grandes retos lógicos:

- Generación de Entornos Válidos: Construir un escenario aleatorio no trivial que garantice la existencia de un camino continuo entre la entrada y la salida, evitando callejones cerrados por completo o zonas inalcanzables.
- Navegación y Optimización: Encontrar la ruta más corta entre el inicio y la meta dentro de una red de caminos posibles sin caer en ciclos infinitos.

3. Justificación de los Algoritmos Seleccionados

- DFS (*Depth-First Search*) para la Generación: Se eligió DFS porque explora los caminos a la máxima profundidad posible y utiliza *backtracking* (retroceso) mediante una Pila (Stack). Esta lógica es ideal para esculpir laberintos, ya que genera pasillos sinuosos y callejones complejos, garantizando un mapa conexo y sin zonas inalcanzables.
- BFS (*Breadth-First Search*) para la Resolución: Se eligió BFS porque evalúa el mapa de forma concéntrica por niveles o capas mediante una Cola (Queue). Al explorar todas las rutas posibles simultáneamente a la misma distancia, garantiza matemáticamente encontrar la ruta más corta desde la entrada hasta la salida, algo que DFS no puede asegurar.

4. Objetivos

Objetivo General

Desarrollar un programa en lenguaje C que genere laberintos aleatorios mediante un algoritmo de búsqueda en profundidad (DFS) y permita su resolución tanto manual por parte del usuario como automática mediante una búsqueda en anchura (BFS).

Objetivos Específicos

- Implementar una estructura de matriz bidimensional para la representación digital del laberinto, gestionando eficientemente las coordenadas de paredes y caminos.
- Dominar la lógica del algoritmo DFS (Depth-First Search) modificado para la demolición de paredes y creación de caminos conectados.
- Diseñar una interfaz de usuario interactiva por teclado que capture los movimientos del jugador en tiempo real.
- Aplicar el algoritmo BFS (Breadth-First Search) para garantizar el hallazgo de la ruta más corta desde el inicio hasta la salida del laberinto.

5. ¿En qué consiste el programa?

El programa es un simulador interactivo de laberintos basado en texto/consola. Su núcleo se divide en tres componentes principales:

1. El Tablero (La Matriz): El entorno se modela mediante una matriz bidimensional de tamaño $N \times M$. Dentro de esta matriz, el programa interpreta binariamente el espacio: los valores 1 representan paredes infranqueables y los valores 0 representan caminos transitables.
2. El Modo Jugador (Resolución Manual): Una vez generado el laberinto, el usuario toma el control de un avatar/puntero y, mediante teclas físicas (W, A, S, D), intenta navegar a través de las casillas libres desde la salida hasta la meta.
3. El Modo Algoritmo (Resolución Automática): Al finalizar la partida manual, el programa activa un agente automático que calcula e imprime sobre la pantalla los pasos exactos y óptimos para resolver el laberinto.

6. Modo de Juego y Resolución Manual

Una vez que el algoritmo DFS construye el laberinto, se inicia la fase interactiva donde el usuario toma el control directo del personaje en tiempo real:

- Controles de Movimiento: El usuario utiliza las teclas estándar de navegación mediante la consola:
 - W: Moverse hacia arriba.
 - S: Moverse hacia abajo.
 - A: Moverse hacia la izquierda.
 - D: Moverse hacia la derecha.
- Mecánica de Juego y Validación: Tras ingresar una tecla y presionar Enter, el programa valida si la casilla de destino en la matriz tiene el valor 0 (camino libre) o la casilla S (salida). Si es un movimiento válido, la posición del jugador (P) se actualiza. Si el destino contiene un 1 (pared #), el sistema bloquea el paso y el personaje permanece en su lugar.

- Condición de Victoria: La fase manual concluye exitosamente cuando las coordenadas del jugador coinciden con las coordenadas fijadas para la Salida (S).

7. Modo de Resolución Automática (BFS)

Inmediatamente después de completar el recorrido manual (o al avanzar a la siguiente fase), el programa demuestra el cálculo mediante el algoritmo BFS (Breadth-First Search):

- Exploración por Niveles: El algoritmo inicia en la Entrada (E) y utiliza una estructura de Cola (Queue). Revisa todas las casillas adyacentes con valor 0 a una distancia de 1 paso, luego a 2 pasos, y así sucesivamente de forma concéntrica.
- Registro de Trazabilidad: Cada vez que la Cola visita un nuevo espacio, guarda en memoria cuál fue la casilla "padre" que lo llevó ahí. Esto evita que el algoritmo entre en bucles y permite reconstruir el camino exacto.
- Trazado de la Ruta Óptima: Al tocar la Salida (S), BFS detiene la búsqueda. Utilizando la matriz de padres, retrocede desde la salida hasta la entrada cambiando el valor en la matriz de esas casillas a 2.
- Visualización Final: El programa limpia la pantalla y redibuja el laberinto, mostrando la ruta mínima garantizada dibujada con asteriscos (*).

8. Estructuras de Datos y Representación del Mapa

La información del mapa se representa en memoria mediante una matriz bidimensional codificada de la siguiente forma:

Valor en Memoria	Representación Visual	Significado
1	#	Pared infranqueable
0	.	Camino libre / transitable
2	*	Ruta óptima trazada por el algoritmo BFS
Coordenada Inicio	E / P	Entrada / Posición del Jugador
Coordenada Fin	S	Salida / Meta del laberinto

9. Conclusiones

El desarrollo de este proyecto demostró cómo la combinación de estructuras de datos estándar (matrices, pilas y colas) y algoritmos de búsqueda permite resolver problemas complejos de navegación en entornos discretos.

- DFS es el algoritmo adecuado para el diseño procedimental de mapas gracias a su comportamiento exploratorio inductivo al retroceso (*backtracking*).
- BFS es la solución óptima para la búsqueda de caminos en grafos no ponderados, superando en eficiencia y precisión la toma de decisiones manual realizada por un

usuario.

- El lenguaje C ofrece la estructura de control de memoria e iteración matricial idónea para procesar estas transformaciones de datos en tiempo real de manera liviana y eficiente.